```
"""
Functional test runner for the Daily Expense Tracker (Week 4 task).
It runs the program with prepared inputs, compares the output with the
expected result and prints PASS or FAIL for every test case.

Usage:  py test_expense_tracker.py                 (tests expense_tracker_week2.py)
        py test_expense_tracker.py expense_tracker_improved.py
"""
import os
import subprocess
import sys
import tempfile

SCRIPT = os.path.abspath(sys.argv[1] if len(sys.argv) > 1 else "expense_tracker_week2.py")

# Each test: id, description, inputs typed by the user, a check function
# that decides PASS or FAIL, a short text describing the expected result,
# and optional old data placed in expenses.csv before the program starts.
TESTS = [
    ("TC01", "Normal entries", "Food\n100\nTravel\n50\nq\n",
     lambda o, e: "Total spent: 150.0" in o and "Money left in budget: 4850.0" in o,
     "Total 150.0 and 4850.0 left", None),
    ("TC02", "Category typed in small letters", "travel\n50\nq\n",
     lambda o, e: "Travel: 50.0" in o,
     "Saved as Travel", None),
    ("TC03", "Category typed with extra spaces", "  food  \n20\nq\n",
     lambda o, e: "Food: 20.0" in o,
     "Saved as Food", None),
    ("TC04", "Letters instead of an amount", "Food\nabc\nq\n",
     lambda o, e: "Please enter a number" in o and "Total spent: 0" in o,
     "Friendly message, nothing saved", None),
    ("TC05", "Amount is zero", "Food\n0\nq\n",
     lambda o, e: "Amount must be more than 0" in o,
     "Message: amount must be more than 0", None),
    ("TC06", "Negative amount", "Food\n-50\nq\n",
     lambda o, e: "Amount must be more than 0" in o,
     "Message: amount must be more than 0", None),
    ("TC07", "Total above the budget", "Bills\n6000\nq\n",
     lambda o, e: "Budget limit crossed" in o,
     "Budget warning is shown", None),
    ("TC08", "Total exactly equal to the budget", "Bills\n5000\nq\n",
     lambda o, e: "Money left in budget: 0.0" in o and "Warning" not in o,
     "0.0 left and no warning", None),
    ("TC09", "Quit without entering anything", "q\n",
     lambda o, e: "Total spent: 0" in o and "Money left in budget: 5000" in o,
     "Total 0 and 5000 left", None),
    ("TC10", "Decimal amounts", "Food\n99.5\nFood\n0.5\nq\n",
     lambda o, e: "Food: 100.0" in o,
     "Food total 100.0", None),
    ("TC11", "Amount typed with a comma (1,000)", "Food\n1,000\nq\n",
     lambda o, e: "Please enter a number" in o,
     "Friendly message (comma is not a number)", None),
    ("TC12", "Empty category (only Enter pressed)", "\n100\nq\n",
     lambda o, e: "Saved!" not in o,
```

```
     "Entry rejected", None),
    ("TC13", "Number typed as the category", "50\n100\nq\n",
     lambda o, e: "Saved!" not in o,
     "Entry rejected", None),
    ("TC14", "Amount typed as nan", "Food\nnan\nq\n",
     lambda o, e: "Saved!" not in o,
     "Entry rejected", None),
    ("TC15", "Amount typed as inf", "Food\ninf\nq\n",
     lambda o, e: "Saved!" not in o,
     "Entry rejected", None),
    ("TC16", "Damaged line in expenses.csv", "q\n",
     lambda o, e: "Traceback" not in e,
     "Program starts without crashing", "2026-10-01,Food\n"),
]


def run_case(inputs, old_data):
    with tempfile.TemporaryDirectory() as folder:
        if old_data:
            with open(os.path.join(folder, "expenses.csv"), "w") as f:
                f.write(old_data)
        result = subprocess.run([sys.executable, SCRIPT], input=inputs, text=True,
                                capture_output=True, cwd=folder, timeout=20)
        return result.stdout, result.stderr


def key_lines(out, err):
    if "Traceback" in err:
        return "Crash: " + err.strip().splitlines()[-1]
    out = out.replace("Category (Food/Travel/Bills/Fun/Other) or q: ", "").replace("Amount: ", "")
    keep = ("Total spent", "Money left", "Warning", "Please enter", "Amount must", "Saved!",
"Skipped")
    lines = [l.strip() for l in out.splitlines() if any(k in l for k in keep)]
    saved = lines.count("Saved!")
    lines = [l for l in lines if l != "Saved!"]
    if saved:
        lines.insert(0, f"Saved! x{saved}")
    return " | ".join(lines[:4]) if lines else "(no output)"


def persistence_test():
    """TC17: data from the first run must still be there in the second run."""
    with tempfile.TemporaryDirectory() as folder:
        for text in ("Food\n100\nq\n", "Travel\n50\nq\n"):
            r = subprocess.run([sys.executable, SCRIPT], input=text, text=True,
                               capture_output=True, cwd=folder, timeout=20)
        ok = "Total spent: 150.0" in r.stdout
        return ok, key_lines(r.stdout, r.stderr)


def main():
    rows = []
    for tid, desc, inputs, check, expected, old in TESTS:
        out, err = run_case(inputs, old)
        status = "PASS" if check(out, err) else "FAIL"
        rows.append((tid, desc, expected, key_lines(out, err), status))
    ok, actual = persistence_test()
    rows.append(("TC17", "Data is kept between two runs", "Total 150.0 in the second run",
```

```
                actual, "PASS" if ok else "FAIL"))
    print("Testing:", os.path.basename(SCRIPT))
    for r in rows:
        print(f"{r[0]}  {r[4]}  {r[1]}")
        print(f"      expected: {r[2]}")
        print(f"      actual  : {r[3]}")
    passed = sum(1 for r in rows if r[4] == "PASS")
    print(f"\nResult: {passed} passed, {len(rows) - passed} failed, {len(rows)} total")
    if os.environ.get("DUMP"):
        import json
        json.dump(rows, open(os.environ["DUMP"], "w"))


main()
```